QUESTIONS 10 / 10 completed

Polynomials

POLYNOMIALS

"In scientific computing applications, mathematical polynomials are often represented using linked lists where each node stores a coefficient and exponent. Write a C program to represent two polynomials using

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

// Node structure for polynomial term
struct Node {
    int coeff;
    int exp;
    struct Node *next;
};

// Create a new term
struct Node* createNode(int coeff, int exp) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->coeff = coeff;
    newNode->exp = exp;
    newNode->next = NULL;
    return newNode;
}

// Insert term at end
void insert(struct Node** head, int coeff, int exp) {
```

INPUT

Polynomial 1: $5x^2 + 3x + 2$
Polynomial 2: $4x^2 + 6x + 1$

QUESTIONS 10 / 10 completed

Single Linked List

SINGLE LINKED LIST

"A college maintains a list of student roll numbers in the order they are registered. New students may join at different stages of the registration process. Develop a C program using a singly linked list to store the roll numbers and perform insertion

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *head = NULL;

// Display the list
void display() {
    struct Node *temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Insert at beginning
```

INPUT

```
Initial List: Empty
Insert 10 at beginning
Insert 20 at end
Insert 15 at position 2
```

QUESTIONS 10 / 10 completed

Deletion Operatior

DELETION OPERATIONS

"A company maintains employee records in a linked list. Employees may leave the organization, requiring their records to be removed. Write a C program to implement deletion operations from the beginning, end, and a specified position

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

// Structure for a node
struct Node {
    int data;
    struct Node *next;
};

// Insert node at end
void insert(struct Node **head, int data) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node *temp = *head;
```

INPUT

10 -> 20 -> 30 -> 40
Delete from beginning

QUESTIONS 10 / 10 completed

Linked List

LINKED LIST

"A library stores book identification numbers in a linked list. Develop a C program that searches for a given element and displays its position if found.
Test Case 1

Input

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

// Structure for a node
struct Node {
    int data;
    struct Node *next;
};

// Insert node at end
void insert(struct Node **head, int data) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node *temp = *head;
```

INPUT

30

QUESTIONS 10 / 10 completed

Singly Linked List-F

SINGLY LINKED LIST-REVERSE

"In a logistics application, delivery locations are stored in a linked list. Sometimes the route needs to be traversed in reverse orderWrite a C program to reverse a singly linked list and display the reversed list

Test Case 1

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node *next;
};

// Insert node at end
void insert(struct Node **head, int data) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node *temp = *head;
```

INPUT

4
10 20 30 40

QUESTIONS 10 / 10 completed

Singly Linked List-N

SINGLY LINKED LIST-NODES

"A hospital maintains patient records using a linked list. To generate reports, the system needs to determine the total number of records currently stored.Write a C program to count and display the total number of nodes in a singly linked list.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

// Structure for a node
struct Node {
    int data;
    struct Node *next;
};

// Function to insert a node at the end
void insert(struct Node **head, int data) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node *temp = *head;
```

INPUT

5
10 20 30 40 50

QUESTIONS 10 / 10 completed

Single Sorted Linke

SINGLE SORTED LINKED LIST

"Two departments maintain separate sorted lists of student IDs. For centralized processing, both lists need to be merged while preserving the sorted order.Write a C program to merge two sorted singly linked lists into a single sorted linked list.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

// Structure for a node
struct Node {
    int data;
    struct Node *next;
};

// Create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Insert at end
void insert(struct Node** head, int data) {
    struct Node* newNode = createNode(data);

```

INPUT

List 1: 10 -> 30 -> 50
List 2: 20 -> 40 -> 60

QUESTIONS 10 / 10 completed

Sorted Linked List-

SORTED LINKED LIST-2

"A database contains sorted customer IDs, but due to data-entry errors, duplicate records have been created. Develop a C program that removes duplicate nodes from a sorted linked list and displays the updated list.

Test Case 1

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node *next;
};

// Create a new node
struct Node* createNode(int data) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Insert node at end
void insert(struct Node **head, int data) {
    struct Node *newNode = createNode(data);

```

INPUT

6
10 10 20 20 30 30

QUESTIONS 10 / 10 completed

Circular Linked List

CIRCULAR LINKED LIST

"In a round-robin CPU scheduling system, processes are executed cyclically. Such applications can be represented using a circular linked list. Write a C program to create a circular linked list and display all nodes by traversing the list exactly once.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node *next;
};

// Insert node into circular linked list
void insert(struct Node **head, int data) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = data;

    if (*head == NULL) {
        *head = newNode;
        newNode->next = *head;
        return;
    }

    struct Node *temp = *head;
```

INPUT

4
10 20 30 40

QUESTIONS 10 / 10 completed

Doubly Linked List

DOUBLY LINKED LIST

"A music player maintains a playlist where users can move both forward and backward between songs. This can be efficiently implemented using a doubly linked list. Write a C program to create a doubly linked list and display nodes

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

// Structure for a doubly linked list node
struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};

// Create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;
    return newNode;
}

// Insert node at the end
void insert(struct Node** head, int data) {
```

INPUT

4
10 20 30 40